Informed Search:

Best first search
Node is selected depending on the function $f(n)$ evaluation function
(Difference from the general → is that in the queue)

1 Greedy Best first search

$f(n) = h(n)$ ← estimate

$O(b^m)$

cheapest and closest to the goal

m is the max depth of the search space

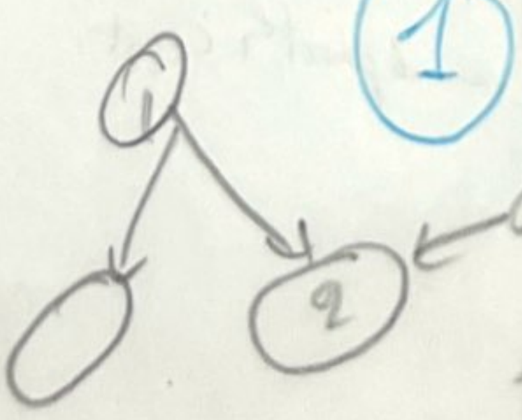
Am i lead to quickly solution.

like straight line distance.

Use the distance not the actual step cost !!

Not optimal + incomplete.

even if finite state spaces like DFS.

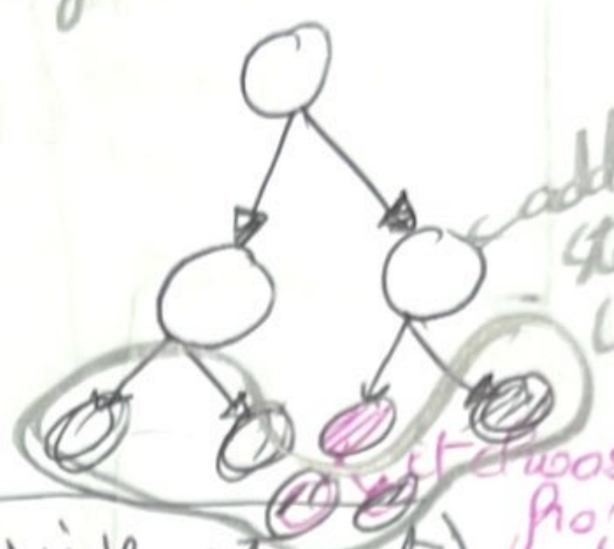


consider only the estimate from the node to the goal.

2 A* search

$f(n) = g(n) + h(n)$

estimated cost of the cheapest solution.



add the step cost + estimate

it chooses from the best one in the frontier queue ($f(n)$)

Where $h(n)$ is the estimate

$h(n) \leq c(n, a, n') + h(n')$ ← admissible. ← consistency

consistent + admissible

consistent → admissible.

Graph optimal when it is consistent

- for all nodes:

$f(n) \leq C^*$

but

- all $f(n) < C^*$ nodes

might be $f(n) = C^*$

$\varepsilon > 0$

finite + ε finite → complete.

No Nodes with $f(n) > C^*$ pruning subtrees. Don't go to it

- always expands fewer nodes than other algorithms.

COMPLETE + OPTIMAL.

The number of states with $h(n) < C^* - \varepsilon$ is still exponential in the length of the solution.

Tells us how close to optimal I can be.

3 Memory bounded heuristic search

IDA*

- Cutoff based on $f(\text{cth})$
- practical for many nodes with unit step cost

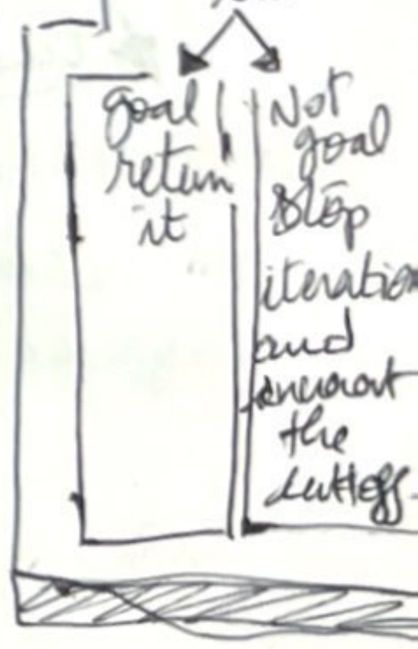
RBFS

(recursion when we backtrack)

- linear space.
- Current limit the recursion unwinds back to alternative

Not necessarily from the states expanded from the node.

f:



Cont'd of

RBFS:

- recursion unwinds → RFBS replaces the f-value of each node along the path with ↗ Backed up Value - the best Value of its children.

- Suffers from excessive node regeneration
- More efficient IDA*

h(n) admissible → optimal.

★ linear complexity space in the depth of the deepest optimal solution.

O(bd)

★ time complexity: dependson accuracy of h(n) how often the best path changes as nodes are expanded.

but it is potentially exponential.

(2)

Learning to search

★ using meta level state space

Where each state is a tree because each meta-level state space it captures the internal computational state of a program searching a object-level-state space.

like romania.

search tree.

• and each action here is a computational step that alters the internal state.

• like in A* → computational step expanding a leaf node and adding its successor to the tree.

METALEVEL ★ LEARNING

it can learn from these experiences to avoid exploring unpromising subtrees.

Goal is to minimize the total cost of problem solving. and trading off computational expense and path cost.

Heuristic Functions:

- for 15-Puzzle:

To use A* ⟶ admissible $h(n)$.

h_1:
- the number of misplaced tiles.

it is admissible because we need at least one step for each misplaced one to get it to its correct position.

[NEVER OVERestimate].

h_2: ✧
the sum of the distances of the tiles from ✧ their goal positions

↓

(moves are only horizontal or vertical, the function is then called block distance or manhattan distance).

Heuristic because it always takes the minimum number of steps needed to reach the goal for any given state.

✧✧

Heuristic accuracy and performance:

effective branching factor b^*

- solution depth is d with N nodes generated then b^* is the b that a uniform tree of depth d would have in order to contain N+1 nodes.

$N+1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$.

Exemple for { Nodes = 52, d = 5

then $\frac{b^{*d+1} - 1}{b^* - 1} = N+1$

Solve this equation.

$b^{*d+1} - 1 = (N+1)(b^* - 1)$

Solve!! $b^{*6} - 1 = N b^* - N + b^* - 1$

$\rightarrow b^{*6} - N b^* + b^* = N \longrightarrow b^* (b^*$

h_2 is better
- We notice by nodes: $h_2(n) \geq h_1(n)$
  ⟶ h_2 dominates h_1.

Domination = efficiency.

$h_1 \dots h_m$ are admissible

So if none dominate the other we take:

$h(n) = \max\{h_1(n), \dots, h_m(n)\}$ (3)

admissible and h is consistent

Learning.

heuristics by noticing the actual cost and solution path

Learn from that point.

from this we can use these examples to learn a model of a function h that can predict solution costs for other states that arise during search.

it can be achieved using: Neural networks.
Decision trees
Reinforcement learning

Inductive Learning.

- Where we; the features of a state that are relevant to predicting the state value.

like: x_1 and x_2 are helpful in predicting the actual distance of state from the goal.

$x_1(n)$ number of misplaced

$x_2(n)$ number of pairs that are not adjacent in the goal state.

taking

$h(n) = c_1 x_1(n) + c_2 x_2(n)$

find the best fit to the actual solution